



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente.

División de Ciencias de la Ingeniería.

Sistema de Base de Datos 2

Ing. Daniel Gonzales.

Aux: Luis Emilio

Respaldos y Recuperación en Bases de Datos Relacionales

SGBD: MySQL 8.0

Manual Técnico

Estudiante	Carné
Byron Fernando Torres Ajxup	201731523

Quetzaltenango, Guatemala.

Lunes 19 de agosto de 2026.

Introducción

Este documento presenta el diseño, la implementación y la administración de una base de datos relacional para gestionar las operaciones de una cadena hotelera. El trabajo se centró en aplicar respaldos completos e incrementales sobre MySQL, comparar distintas formas de recuperar la base ante una falla y medir, en un entorno controlado, cuánto tiempo y espacio requiere cada estrategia.

El contenido sigue el mismo orden en que se realizó el trabajo: primero el diseño e implementación de la base de datos, luego la preparación del mecanismo de respaldo incremental con Binary Logs, después la ejecución día a día de las cargas con sus respaldos correspondientes y, al final, el análisis comparativo de resultados junto con las conclusiones.

Diseño de la Base de Datos

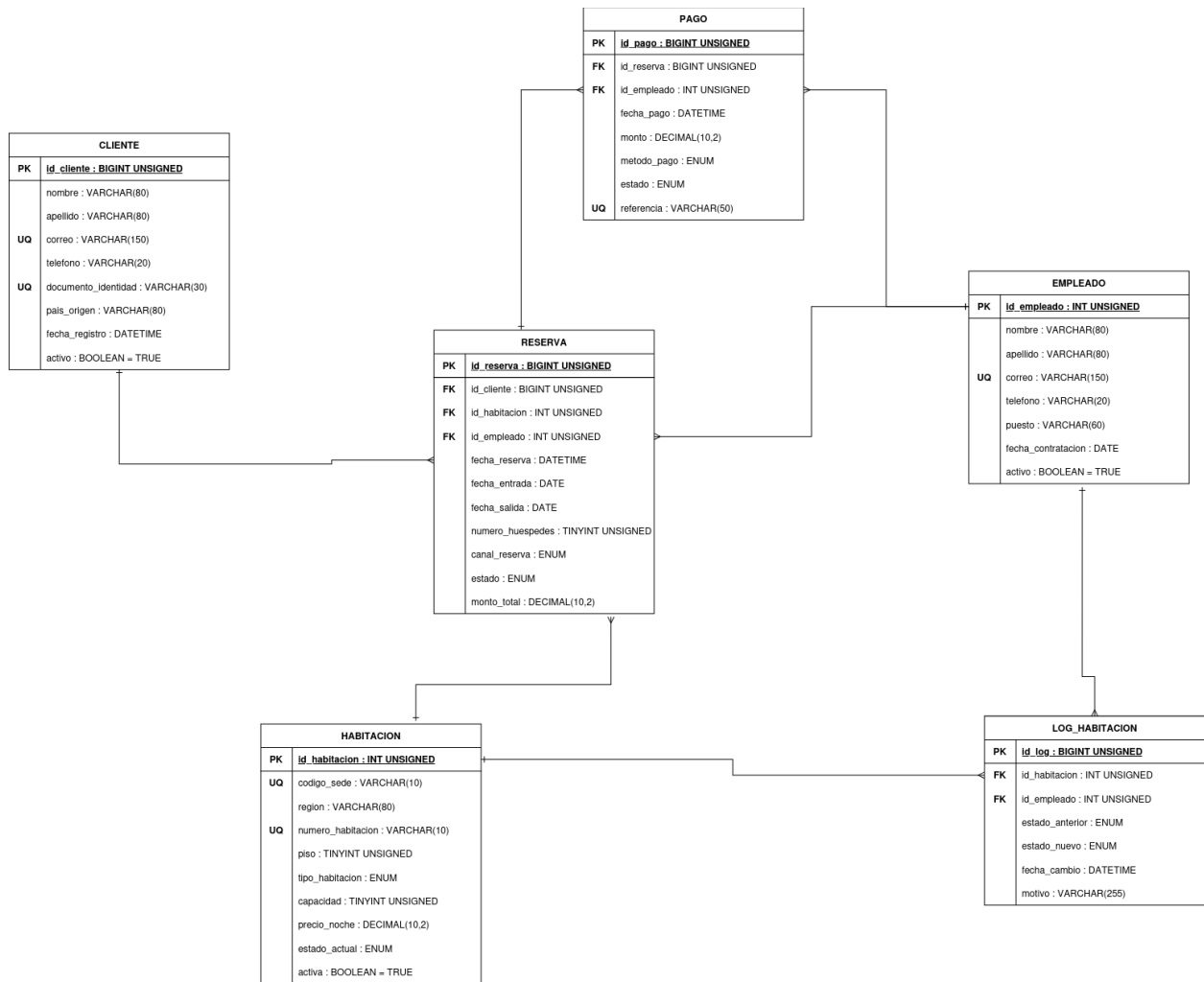
La base de datos cubre las operaciones principales de una cadena hotelera: clientes, habitaciones, empleados, reservas, pagos e historial de cambios de estado de las habitaciones. Se mantuvieron únicamente las seis entidades pedidas en la práctica; se decidió no agregar entidades adicionales como HOTEL, REGION, PUESTO o METODO_PAGO porque no eran necesarias para los objetivos del trabajo y solo habrían complicado la carga, el respaldo y la restauración sin aportar nada a cambio.

Propósito de las entidades

Entidad	Propósito
CLIENTE	Datos generales y de contacto de los clientes.
HABITACION	Características, ubicación y estado de cada habitación.
EMPLEADO	Empleados responsables de registrar operaciones.
RESERVA	Reservaciones realizadas por los clientes.
PAGO	Pagos asociados a una reservación.
LOG_HABITACION	Historial de cambios de estado de las habitaciones.

Diagrama entidad-relación

El diagrama entidad-relación representa las seis entidades, sus atributos principales y las relaciones establecidas mediante claves primarias y foráneas.



Diccionario de datos

Se detallan a continuación las dos entidades núcleo del modelo; el resto sigue el mismo criterio de nomenclatura y restricciones.

CLIENTE

Campo	Tipo	Restricciones	Descripción
id_cliente	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador único.
nombre / apellido	VARCHAR(80)	NOT NULL	Nombre y apellido del cliente.
correo	VARCHAR(150)	NOT NULL, UNIQUE	Correo electrónico.
telefono	VARCHAR(20)	NOT NULL	Número telefónico.
documento_identidad	VARCHAR(30)	NOT NULL, UNIQUE	Documento de identidad.
pais_origen	VARCHAR(80)	NOT NULL	País de origen.
fecha_registro	DATETIME	NOT NULL	Fecha y hora de registro.
activo	BOOLEAN	NOT NULL, DEFAULT TRUE	Conserva clientes históricos sin eliminarlos.

Justificación: además de los datos básicos de contacto, se incorporan atributos para identificar individualmente a los clientes, registrar su procedencia y conservar el historial sin eliminar registros usados por otras operaciones.

HABITACION

Campo	Tipo	Restricciones	Descripción
id_habitacion	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador interno.
codigo_sede	VARCHAR(10)	NOT NULL	Código de la sede.
region	VARCHAR(80)	NOT NULL	Región de la sede.
numero_habitacion	VARCHAR(10)	NOT NULL	Número visible.
piso	TINYINT UNSIGNED	NOT NULL	Piso donde está ubicada.
tipo_habitacion	ENUM	NOT NULL	Tipo de habitación.

Campo	Tipo	Restricciones	Descripción
capacidad	TINYINT UNSIGNED	NOT NULL	Capacidad máxima de huéspedes.
precio_noche	DECIMAL(10,2)	NOT NULL	Precio por noche.
estado_actual	ENUM	NOT NULL, DEFAULT DISPONIBLE	Estado actual.
activa	BOOLEAN	NOT NULL, DEFAULT TRUE	Disponibilidad para operaciones.

Existe una restricción única sobre (codigo_sede, numero_habitacion).

Implementación de la Base de Datos

Con el modelo relacional ya definido, se pasó a implementar la base en MySQL 8.0 usando scripts SQL versionados en el repositorio: 00_create_database.sql, 01_schema.sql y 02_drop_database.sql.

A lo largo de toda la práctica se trabajó con tres herramientas: el cliente mysql para ejecutar scripts y consultas, mysqldump para generar el respaldo base y los respaldos completos, y mysqlbinlog para leer el contenido del Binary Log y extraer de él los respaldos incrementales. Todos los scripts SQL y de carga se mantienen versionados dentro del repositorio, junto con los respaldos generados y las evidencias de cada etapa.

Creación de la base de datos

00_create_database.sql crea la base hotel_db con el conjunto de caracteres utf8mb4:

```
CREATE DATABASE IF NOT EXISTS hotel_db
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE hotel_db;
```

Ejecución: `mysql -u root -p < sql/00_create_database.sql`

Creación del esquema

01_schema.sql define las seis tablas (CLIENTE, HABITACION, EMPLEADO, RESERVA, PAGO, LOG_HABITACION), junto con claves primarias, claves foráneas, restricciones UNIQUE y CHECK, valores por defecto y reglas de integridad referencial. Ejecución: `mysql -u root -p < sql/01_schema.sql`

Reconstrucción de la base de datos

Durante el desarrollo fue necesario sustituir una versión inicial del esquema por el diseño definitivo, eliminando la base con 02_drop_database.sql (DROP DATABASE IF EXISTS hotel_db;) y recreándola desde los scripts versionados:

```
mysql -u root -p < sql/02_drop_database.sql  
mysql -u root -p < sql/00_create_database.sql  
mysql -u root -p < sql/01_schema.sql
```

Preparación de los Respaldos Incrementales

Para los respaldos incrementales se utilizaron los Binary Logs de MySQL, que registran los cambios realizados sobre la base de datos y permiten reproducirlos posteriormente.

Verificación de Binary Logs

Se verificó que el registro binario estuviera habilitado:

```
mysql -u root -p -e "SHOW VARIABLES LIKE 'log_bin'; SHOW MASTER STATUS; SHOW BINARY LOGS;"
```

Resultado: log_bin = ON. Binary Log utilizado durante la carga del Día 1: binlog.000006.

También se confirmó binlog_format = ROW, que registra los cambios a nivel de fila.

Backup Base

Antes de construir la cadena de respaldos incrementales es necesario contar con un respaldo que contenga únicamente la estructura inicial:

```
mysqldump -u root -p --no-data --routines --triggers \  
hotel_db > backups/backup_base.sql
```

La opción `--no-data` exporta únicamente la estructura, sin registros. Esto se confirmó con `grep -m 3 "^INSERT INTO" backups/backup_base.sql`, que no devolvió resultados. Archivo: `backups/backup_base.sql` — 8,934 bytes — 2026-08-18 23:45:25.

Procedimiento ideal

En este caso el backup base se generó después de la carga del Día 1; esto no afecta su contenido porque se usó `--no-data`. Lo técnicamente recomendado habría sido seguir este orden: crear la base de datos → crear las tablas → crear el backup base → registrar la posición inicial del Binary Log (`mysql -u root -p -e "SHOW MASTER STATUS;"`) → realizar la carga del Día 1. Esto evita tener que localizar después el inicio de la transacción dentro del Binary Log.

Día 1 — Carga Inicial

El archivo `sql/carga/01_carga_dia1.sql` insertó 300 registros en `CLIENTE`, 300 en `HABITACION` y 300 en `EMPLEADO` (900 en total), ejecutado con `mysql -u root -p < sql/carga/01_carga_dia1.sql`. La verificación con `SELECT COUNT(*)` confirmó 300 registros en cada tabla.

Uso de transacción

El archivo agrupa las inserciones entre `START TRANSACTION;` y `COMMIT;;`, lo que facilitó identificar posteriormente la operación completa dentro del Binary Log.

Backup Completo del Día 1

Al finalizar la carga se generó un respaldo completo:

```
mysqldump -u root -p --single-transaction --routines --triggers \  
hotel_db > backups/full/full_dia1.sql
```

A diferencia del backup base, `grep -m 3 "^INSERT INTO"` sí devolvió resultados, confirmando que contiene estructura y datos. Archivo: `full_dia1.sql` — 91,964 bytes — 2026-08-19 00:06:31. Este respaldo puede restaurar por sí mismo el estado completo del Día 1.

Backup Incremental del Día 1

Como no se registró la posición del Binary Log antes de la carga, fue necesario localizar el intervalo de la transacción del Día 1 usando mysqlbinlog con decodificación de eventos de fila (--base64-output=DECODE-ROWS -vv).

Se identificó el inicio de la transacción (# at 11127 → BEGIN) y, mediante la inspección de los eventos Xid, su fin (# at 71262, end_log_pos 71293, Xid = 94 → COMMIT). El intervalo resultante fue posición inicial = 11127 y posición final = 71293.

Con esas posiciones se generó el incremental:

```
sudo mysqlbinlog --start-position=11127 --stop-position=71293 \  
/var/lib/mysql/binlog.000006 > backups/incremental/incremental_dia1.sql
```

Archivo: incremental_dia1.sql — binlog.000006 — inicio 11127, fin 71293 — 84,326 bytes — 2026-08-19 00:36:19. Se verificó con `grep -n -E "BEGIN|COMMIT"` que el archivo conserva el inicio y el fin completos de la transacción.

Resumen del Día 1

Elemento		Resultado	
CLIENTE / HABITACION / EMPLEADO		300 registros cada una (900 en total)	
Backup base / completo / incremental		Correcto	
Binary Log		binlog.000006	
Inicio / fin incremental		11127 → 71293	
Respaldo	Tipo	Tamaño	
backup_base.sql	Base	8,934 bytes	
full_dia1.sql	Completo	91,964 bytes	
incremental_dia1.sql	Incremental	84,326 bytes	

Nota metodológica: durante la ejecución inicial no se obtuvo una medición limpia del tiempo de creación porque el ingreso interactivo de la contraseña alteraba el valor mostrado por time. Al finalizar la práctica se repitieron las mediciones en condiciones controladas (login-path de MySQL), sin sobrescribir los respaldos originales; esos son los valores usados en el análisis.

Procedimiento Recomendado para los Siguientes Días

A partir del Día 2 se usó un procedimiento más controlado: antes de cada carga se registra la posición inicial del Binary Log (SHOW MASTER STATUS;), se ejecuta la carga y, al finalizar, se registra la posición final. El incremental queda definido directamente por ese rango (posición inicial → posición final), sin necesidad de buscar manualmente las transacciones después de ejecutadas:

```
sudo mysqlbinlog --start-position=X --stop-position=Y \  
/var/lib/mysql/binlog.XXXXXXX > backups/incremental/incremental_diaN.sql
```

Para cada etapa se midió el tiempo con time sobre el comando correspondiente (por ejemplo, time mysqldump ... > backups/full/full_diaN.sql), registrando el resultado en la bitácora. El principio aplicado en cada etapa fue: registrar inicio → realizar cambios → registrar fin → crear el backup completo → extraer el incremental → registrar tamaño y tiempo → actualizar la bitácora, manteniendo así una cadena de respaldos reproducible.

Ejecución de las Etapas Día 2 a Día 5

Las etapas posteriores siguieron el mismo procedimiento técnico documentado para el Día 1: (1) registrar la posición inicial del Binary Log, (2) ejecutar el archivo de carga correspondiente, (3) verificar los registros insertados, (4) registrar la posición final del Binary Log, (5) crear el backup completo, (6) extraer el intervalo correspondiente al backup incremental, (7) registrar tamaño, fecha, hora y tiempo de ejecución, y (8) guardar las evidencias correspondientes. A continuación se detallan los resultados reales obtenidos en cada etapa.

Día 2 — RESERVA

- Archivo de carga: sql/carga/02_carga_dia2.sql
- Registros insertados: 300 — Tiempo de carga: 2.935 s
- Backup completo: backups/full/full_dia2.sql — Tamaño: 120,876 bytes — Tiempo de creación: 0.056 s
- Incremental: backups/incremental/incremental_dia2.sql — Binary Log: binlog.000006
- Posición inicial: 71293 — Posición final: 82739 — Tamaño: 18,106 bytes — Tiempo de creación: 0.029 s
- Evidencia: evidencia/dia2/dia2_resumen.png — Resultado: Correcto

Día 3 — Primer LOG_HABITACION

- Archivo de carga: sql/carga/03_carga_dia3.sql
- Registros insertados: 300 — Tiempo de carga: 2.419 s
- Backup completo: backups/full/full_dia3.sql — Tamaño: 145,628 bytes — Tiempo de creación: 0.055 s
- Incremental: backups/incremental/incremental_dia3.sql — Binary Log: binlog.000006
- Posición inicial: 82739 — Posición final: 95746 — Tamaño: 20,221 bytes — Tiempo de creación: 0.017 s
- Evidencia: evidencia/dia3/dia3_resumen.png — Resultado: Correcto

Día 4 — PAGO

- Archivo de carga: sql/carga/04_carga_dia4.sql
- Registros insertados: 300 — Tiempo de carga: 2.603 s
- Backup completo: backups/full/full_dia4.sql — Tamaño: 169,430 bytes — Tiempo de creación: 0.053 s
- Incremental: backups/incremental/incremental_dia4.sql — Binary Log: binlog.000006
- Posición inicial: 95746 — Posición final: 106886 — Tamaño: 17,698 bytes — Tiempo de creación: 0.016 s
- Evidencia: evidencia/dia4/dia4_resumen.png — Resultado: Correcto

Día 5 — Segundo LOG_HABITACION

- Archivo de carga: sql/carga/05_carga_dia5.sql
- Registros insertados: 300 — Total acumulado en LOG_HABITACION: 600
- Tiempo de carga: 3.509 s
- Backup completo: backups/full/full_dia5.sql — Tamaño: 195,494 bytes — Tiempo de creación: 0.051 s
- Incremental: backups/incremental/incremental_dia5.sql — Binary Log: binlog.000006
- Posición inicial: 106886 — Posición final: 120913 — Tamaño: 21,611 bytes — Tiempo de creación: 0.016 s
- Evidencia: evidencia/dia5/dia5_resumen.png — Resultado: Correcto

Resumen consolidado

Día	Entidad cargada	Registros	Tiempo carga	Full (bytes)	Incremental (bytes)
2	RESERVA	300	2.935 s	120,876	18,106
3	LOG_HABITACION (1.º)	300	2.419 s	145,628	20,221
4	PAGO	300	2.603 s	169,430	17,698
5	LOG_HABITACION (2.º) — 600 acum.	300	3.509 s	195,494	21,611

Todas las restauraciones de prueba realizadas sobre estos respaldos resultaron correctas y las evidencias correspondientes se guardaron en evidencia/diaN/ para cada etapa.

Análisis Comparativo de Respaldos Completos e Incrementales

A lo largo de la práctica se generaron cinco respaldos completos y cinco incrementales. Los resultados consolidados quedaron en resultados/resultados_restauracion.csv, y cada respaldo se probó mediante restauración para comprobar que recuperara correctamente el estado correspondiente a cada etapa.

Comparación de tamaños

Día	Full	Tamaño (bytes)	Incremental	Tamaño (bytes)
1	full_dia1	91,964	incremental_dia1	84,326
2	full_dia2	120,876	incremental_dia2	18,106
3	full_dia3	145,628	incremental_dia3	20,221
4	full_dia4	169,430	incremental_dia4	17,698
5	full_dia5	195,494	incremental_dia5	21,611

Total de los cinco respaldos completos: 723,392 bytes. Total de los cinco incrementales: 161,962 bytes. Después del primer incremental (más grande por contener la carga inicial de 900 registros), los incrementales usaron considerablemente menos espacio al almacenar solo los cambios entre etapas.

Tiempos de creación y restauración

Mediciones controladas, sin incluir el tiempo de escritura de la contraseña:

Día	Creación full (s)	Creación incr. (s)	Restauración full (s)	Restauración incr. (s)
1	0.083	0.025	0.183	0.043
2	0.056	0.029	0.187	0.034
3	0.055	0.017	0.194	0.026
4	0.053	0.016	0.197	0.031

Día	Creación full (s)	Creación incr. (s)	Restauración full (s)	Restauración incr. (s)
5	0.051	0.016	0.204	0.024

Promedios de creación: full 0.060 s, incremental 0.021 s. Promedios de restauración: full 0.193 s, incremental individual 0.032 s (el backup base se restauró en 0.127 s). Los incrementales resultan más rápidos de crear y de restaurar de forma individual, ya que solo contienen los cambios de cada etapa. Sin embargo, reconstruir el estado del Día 5 por esta vía implica restaurar el backup base y los cinco incrementales en orden, lo que suma alrededor de 0.285 s, frente a los 0.204 s que toma restaurar full_dia5.sql directamente. Ahí está el compromiso central: los incrementales ahorran tamaño y tiempo por archivo, pero la recuperación final depende de una cadena de operaciones, mientras que el backup completo pesa más pero permite recuperar un estado específico de forma directa.

Dependencias

Los respaldos completos son independientes entre sí: recuperar el estado del Día 4 solo requiere full_dia4.sql. Los incrementales, en cambio, dependen unos de otros — no es posible restaurar el Día 5 usando únicamente incremental_dia5.sql — por lo que se necesita el backup base y todos los incrementales anteriores en orden: backup_base → incremental_dia1 → incremental_dia2 → incremental_dia3 → incremental_dia4 → incremental_dia5. Esta secuencia se probó y reconstruyó correctamente el estado final.

Complejidad

El respaldo completo resultó más sencillo de administrar y restaurar, porque cada archivo trae consigo la estructura y los datos del momento en que se generó. El incremental, en cambio, exigió más control: había que llevar registro del archivo de Binary Log, de las posiciones inicial y final, del orden de los incrementales, del backup base y de conservar todos los archivos anteriores. Por ejemplo, para el Día 5 se usó binlog.000006, posición inicial 106886 y posición final 120913.

Ventajas y desventajas

	Respaldos completos	Respaldos incrementales
Ventajas	Independientes; restauración sencilla; menos pasos; recuperan un estado específico directamente.	Menos espacio tras el respaldo inicial; guardan solo los cambios; eficientes con pocos cambios entre respaldos.
Desventajas	Mayor uso de espacio; su tamaño crece con la información; pueden tardar más en bases grandes.	Dependen del backup base y de incrementales previos; deben restaurarse en orden; administración más compleja; un incremental dañado impide reconstruir estados posteriores.

Resultado de las pruebas de restauración

Todas las restauraciones completas fueron exitosas, y también fue posible reconstruir correctamente la base mediante la cadena incremental. Al finalizar la restauración de incremental_dia5.sql se obtuvo el mismo resultado que restaurando full_dia5.sql directamente:

Tabla	Registros
CLIENTE	300
HABITACION	300
EMPLEADO	300
RESERVA	300
PAGO	300
LOG_HABITACION	600

Recomendación

No conviene apoyarse exclusivamente en un solo tipo de respaldo. Una estrategia combinada — respaldos completos periódicos con incrementales entre cada uno — ofrece un mejor equilibrio entre facilidad de recuperación, uso de almacenamiento y frecuencia de respaldo.

Conclusiones

- 1. La implementación de respaldos completos e incrementales permitió comprobar de forma práctica dos estrategias distintas de recuperación de una base de datos MySQL.
- 2. Los respaldos completos resultaron más sencillos de administrar y restaurar, pues cada archivo contiene de forma independiente el estado completo correspondiente a cada etapa.
- 3. Los respaldos incrementales redujeron considerablemente el espacio utilizado después de la carga inicial, al almacenar únicamente los cambios entre una etapa y la siguiente.
- 4. La recuperación mediante incrementales exige mayor organización: conservar el backup base, todos los incrementales anteriores y respetar el orden de restauración.
- 5. Los Binary Logs de MySQL permitieron identificar y extraer los cambios de cada etapa mediante posiciones específicas dentro de binlog.000006.
- 6. Tanto los respaldos completos como la cadena incremental permitieron recuperar correctamente los datos de cada día.
- 7. La verificación con COUNT(*) confirmó que la restauración no solo terminó sin errores, sino que recuperó exactamente la cantidad de registros esperada en cada tabla.
- 8. Al finalizar la restauración completa e incremental del Día 5 se obtuvo el mismo estado final: 300 clientes, 300 habitaciones, 300 empleados, 300 reservas, 300 pagos y 600 registros en LOG_HABITACION.
- 9. Una estrategia combinada resulta conveniente en un entorno real: respaldos completos periódicos para facilitar la recuperación y respaldos incrementales entre ellos para reducir el almacenamiento.
- 10. Un respaldo no debe considerarse correcto solo por generarse sin errores; es indispensable comprobar su restauración, integridad y capacidad real de recuperación antes de confiar en él.